

Sentiment Analysis of Product Reviews

EDA, Modeling, Advanced Features, and Streamlit Deployment

Sebin George Lubaba Sadiya NP Manasa Muraleedharan.E

Predictive Analytics Project

May 15, 2026

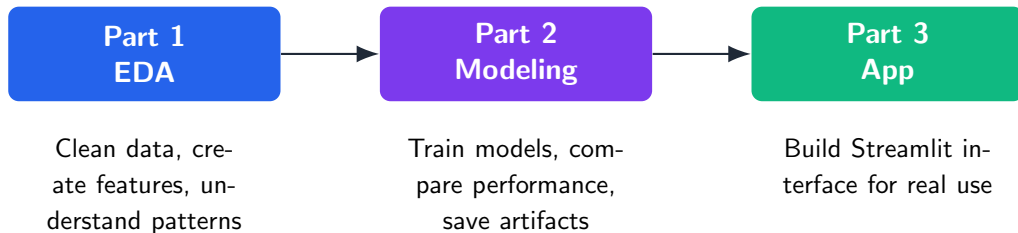
Project background

Customer reviews contain useful information about user satisfaction, product quality, service experience, and customer emotions. Manually reading thousands of reviews is time-consuming, so this project uses Natural Language Processing and machine learning to automatically analyze review text.

Project objective

- Classify reviews as Positive, Neutral, or Negative.
- Understand important patterns in review text using EDA.
- Compare machine-learning and deep-learning models.
- Add advanced features such as emotion detection, aspect-based sentiment analysis, and fake-review detection.
- Deploy the final system using a Streamlit web app.

Project in Three Parts



Explanation: The project starts with data understanding, then moves to model building, and finally becomes an interactive application.

Steps used

- ① Loaded Yelp Review Full dataset.
- ② Converted labels into stars and sentiment classes.
- ③ Created text, punctuation, and VADER features.
- ④ Checked missing values, duplicates, class balance, correlations, and outliers.
- ⑤ Saved final preprocessed data for modeling.

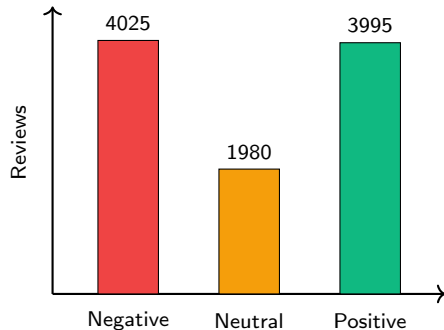
Small explanation: EDA helped us understand the review data before training. It showed that the dataset is clean, but the Neutral class is smaller than Positive and Negative.

EDA Output: Class Distribution

What we found

- Negative: 4,025 reviews.
- Positive: 3,995 reviews.
- Neutral: 1,980 reviews.
- Imbalance ratio: 2.03x.

Explanation: Because Neutral reviews are fewer, the model may find Neutral harder to predict.



Important features created

- word_count
- char_count
- sent_count
- avg_word_len
- caps_ratio
- unique_word_ratio
- vader_compound
- vader_pos
- vader_neg
- vader_neu
- exclamation_density
- review_len_extreme

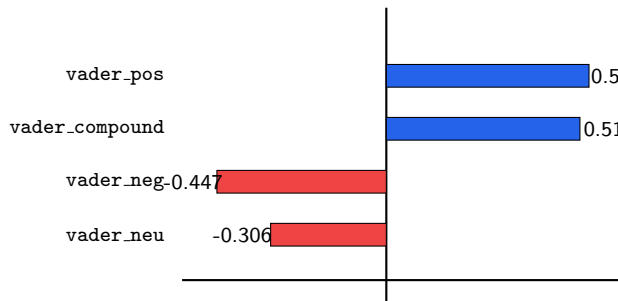
Explanation: These features convert review writing style and sentiment clues into numerical values that machine-learning models can understand.

EDA Output: Top Correlated Features

Key result

- VADER features are strongly related to sentiment.
- vader_pos and vader_compound are most useful.
- Text length features have weaker but useful signals.

Explanation: VADER scores gave a strong baseline signal, so we used them with TF-IDF features during modeling.



Part 2: Modeling Workflow

Steps used

- ① Loaded `preprocessed_data.csv`.
- ② Created 70/15/15 train-validation-test split.
- ③ Converted text into TF-IDF vectors.
- ④ Combined TF-IDF with engineered numerical features.
- ⑤ Trained classical models and DistilBERT.
- ⑥ Compared accuracy, macro F1, speed, and model size.

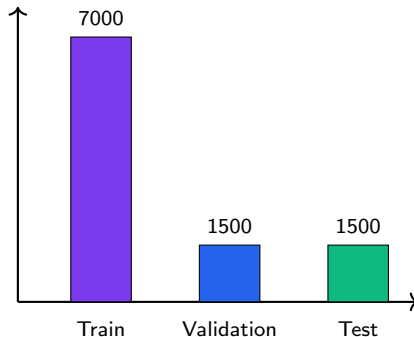
Small explanation: Modeling tested both lightweight machine-learning models and a deep-learning model to decide which approach is best for prediction and deployment.

Modeling: Data Split

Split used

- Training: 7,000 reviews.
- Validation: 1,500 reviews.
- Testing: 1,500 reviews.

Explanation: Stratified splitting keeps the class ratio similar in each split, which makes evaluation fairer.



Modeling: Classical ML Models

Steps used

- TF-IDF created 50,000 text features.
- 13 engineered features were added.
- Final combined feature matrix had 50,013 columns.
- Trained SVM, Logistic Regression, Random Forest, and XGBoost.

Output summary

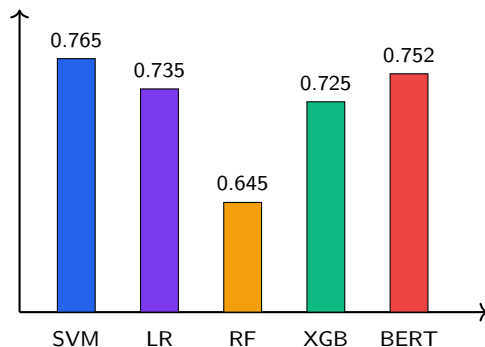
Model	Accuracy	F1 Macro
SVM	0.7653	0.7069
Logistic Regression	0.7347	0.7021
Random Forest	0.6453	0.5937
XGBoost	0.7253	0.6515

Modeling Output: Accuracy Comparison

Main result

- SVM gave the best accuracy.
- Logistic Regression was close.
- XGBoost took much longer to train.
- Random Forest was smallest but less accurate.

Explanation: SVM is the best practical model because it is accurate, fast, and lightweight.



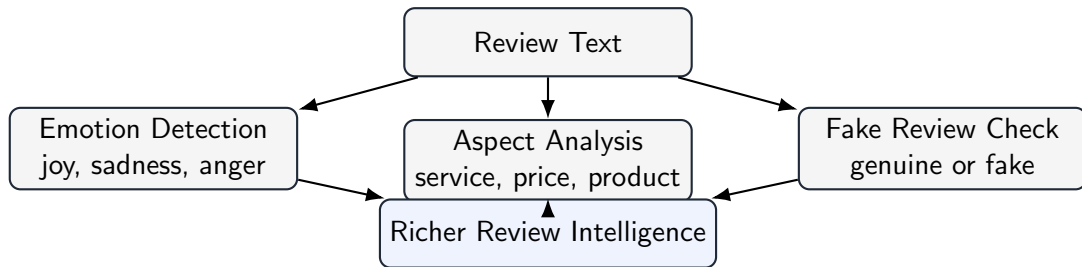
Modeling: DistilBERT

Steps used

- 1 Loaded distilbert-base-uncased.
- 2 Tokenized reviews with maximum length 128.
- 3 Fine-tuned for 3 epochs.
- 4 Saved best checkpoint using validation accuracy.
- 5 Evaluated on the test set.

Metric	Value
Best validation accuracy	0.7613
Test accuracy	0.7520
Test macro F1	0.7148
Training time	4.9 minutes

Advanced Features Added



Explanation: These features make the project more advanced because the app does not only say sentiment; it also explains emotion, review aspects, and authenticity.

Steps used

- Loaded a pre-trained GoEmotions-based DistilBERT model.
- Predicted emotions from review text.
- Displayed emotion probabilities in the app.

Output summary

- Example emotions included anger, sadness, joy, and surprise.
- This helps understand the customer's emotional tone.

Aspect-Based Sentiment Analysis

Steps used

- Created aspect keyword groups.
- Split review into sentences.
- Used VADER score for sentences related to each aspect.
- Returned Positive, Neutral, or Negative for every detected aspect.

Aspect	Meaning
Food/Product	taste, product quality, freshness
Service	staff, waiter, helpfulness, rudeness
Price/Value	expensive, cheap, worth, cost
Ambience	clean, noisy, location, decor
Delivery/Speed	fast, slow, delay, shipping
Battery/Tech	battery, camera, screen, app

Steps used

- Created a rule-based fake-review score.
- Generated pseudo-labels for demonstration.
- Trained a Gradient Boosting classifier.
- Saved fake-review classifier artifact.

Output summary

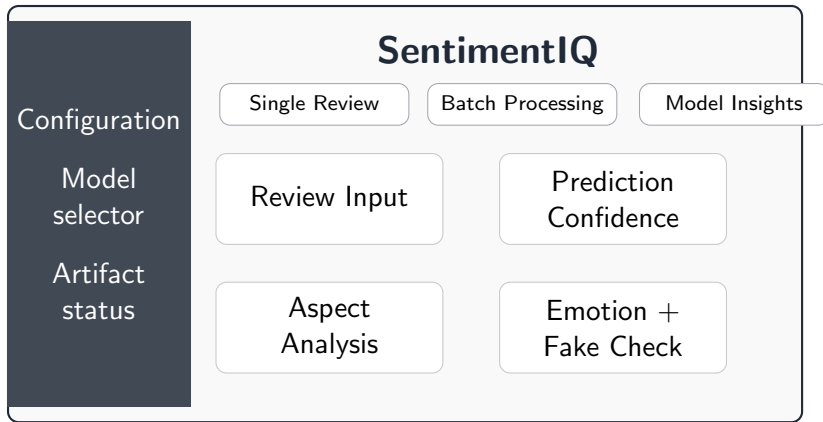
- 6 out of 10,000 reviews were flagged as pseudo-fake.
- This is a demo module; real production use needs labeled fake-review data.

Part 3: Streamlit App Workflow

Steps used

- ① Load saved model artifacts.
- ② User enters one review or uploads a CSV file.
- ③ App cleans text and creates model features.
- ④ Selected model predicts sentiment.
- ⑤ App shows confidence, aspect sentiment, emotion, and fake-review result.

Small explanation: Streamlit converts the notebook work into a usable interface where users can test single reviews or process many reviews together.



Explanation: The app has three main tabs: Single Review, Batch Processing, and Model Insights.

Deployment Steps

Steps used

- 1 Prepare app.py, requirements.txt, and model files.
- 2 Upload files to GitHub.
- 3 Connect repository to Streamlit Community Cloud.
- 4 Select app.py as the main file.
- 5 Deploy and monitor logs.
- 6 Share the public app URL.

Explanation: Deployment makes the project accessible to users without running notebooks manually.

Model choice

- Use SVM for practical deployment because it is accurate, fast, and small.
- Use DistilBERT when contextual understanding is more important.
- Use emotion, ABSA, and fake-review modules to make output more informative.

Final outcome: The project changed from a simple sentiment classifier into a complete review intelligence system.

Conclusion

- Part 1 EDA created a clean and feature-rich dataset.
- Part 2 Modeling compared classical ML and deep learning models.
- Part 3 Streamlit turned the trained models into an interactive app.
- New features make the project stronger: emotion detection, aspect sentiment, and fake-review detection.